SOFE 3200U: Systems Programming

Tutorial 3

Fall 2020

Abdelrahman Elewah

To download the tutorial file, use this command

```
wget https://github.com/elewah/elewah.github.io/blob/master/courses/system%20programming/tutorial3_ppt.pdf
```

In the previous tutorial, we stopped at sort command

Basic Commands

wc

`wc` is a useful command for counting words, lines, or characters. It can receive input from stdin, or a filename as an argument. There are three flags available: `l`, `w`, and `c`; for lines, words, and characters respectively. For example, to display the number of lines and words in some input text, try `printf "Hello\nThis is text" | wc`.

Note: Even if the flags are passed in a different order, wc will always display lines, words, then characters.

sort

`sort` will sort input from stdin, or from a filename accepted as a parameters.

```
sort file > file
```


https://www.geeksforgeeks.org/sort-command-linuxunix-examples/

Agenda

1. Permissions
2. Modifying Ownership and Permissions
3. Makefile
4. Shell scripts

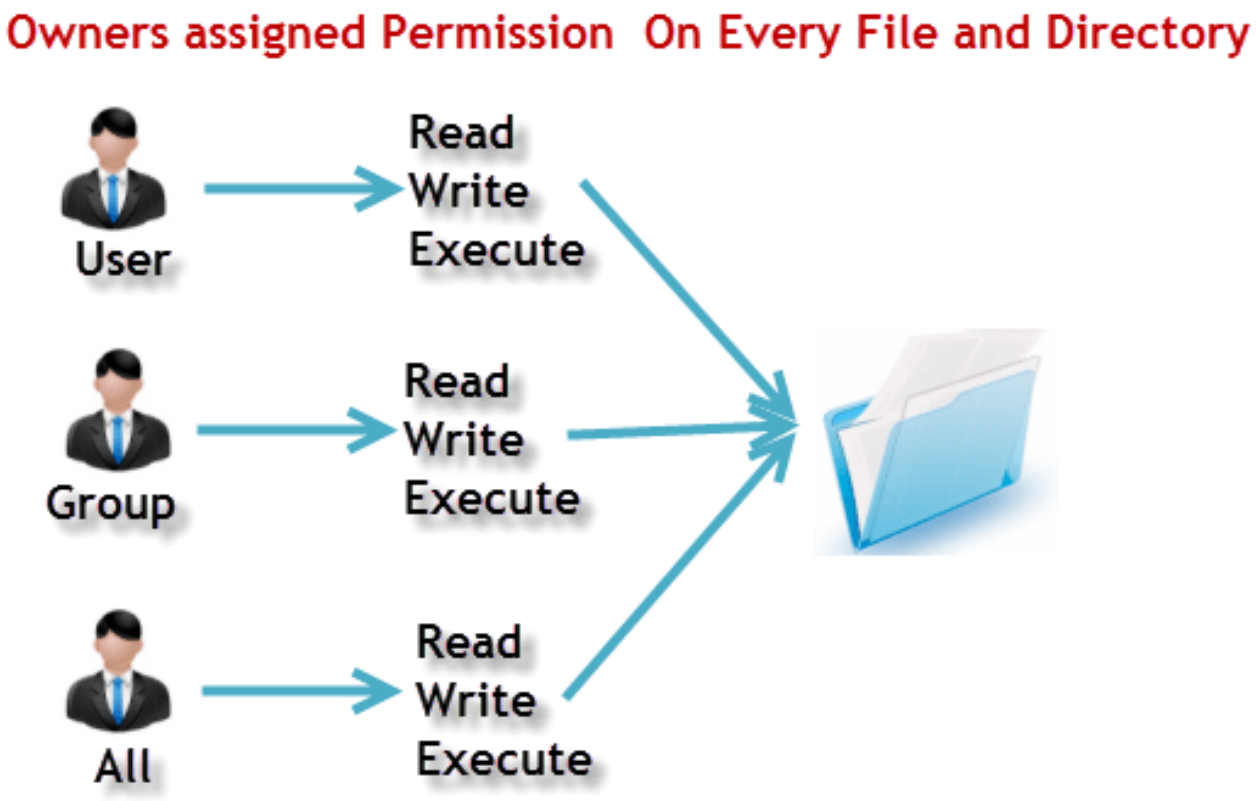
Permissions

Permissions

Ownership of Linux files
Every file and directory on your Unix/Linux system is assigned 3 types of owner, given below.

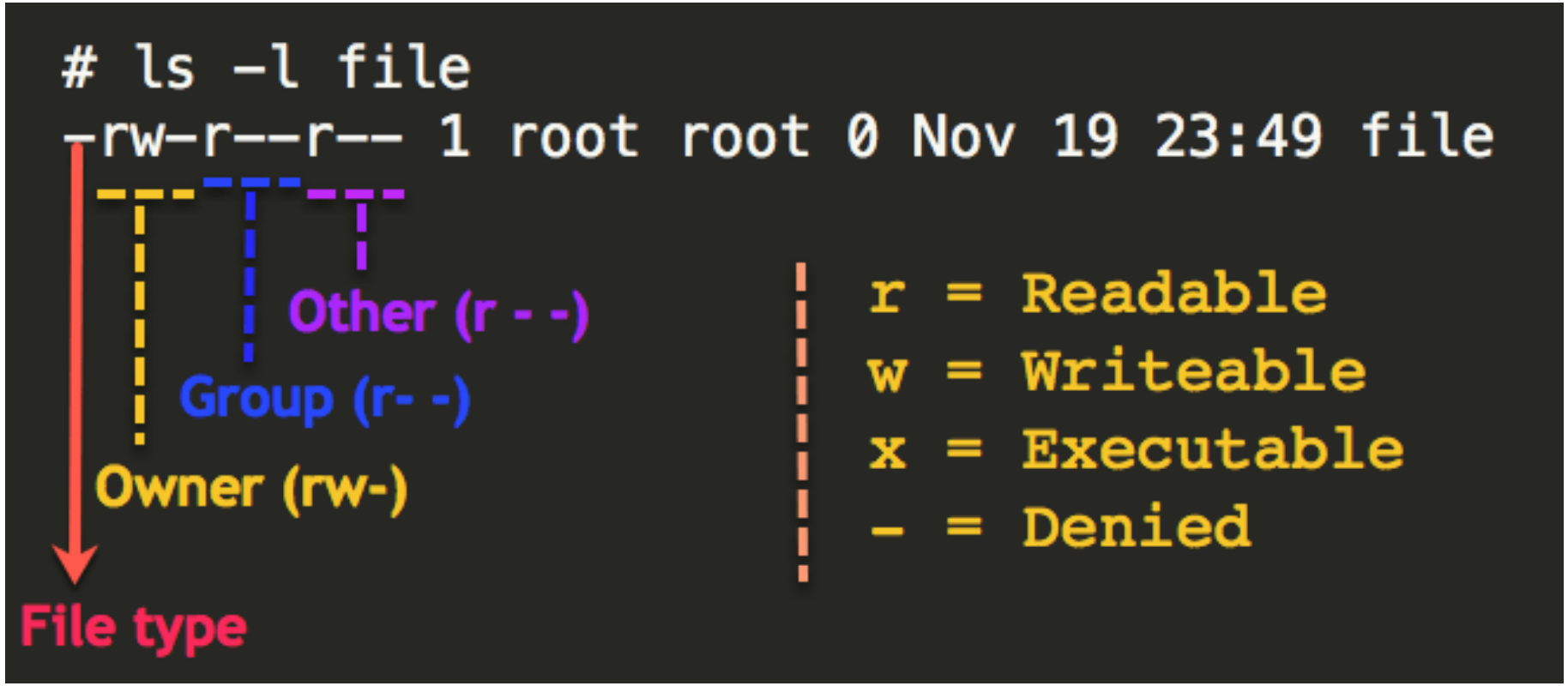
Permissions
Every file and directory in your UNIX/Linux system has 3 permissions (readable, writeable ,executable) defined for all the 3 owners discussed above.

```
ls - l
```



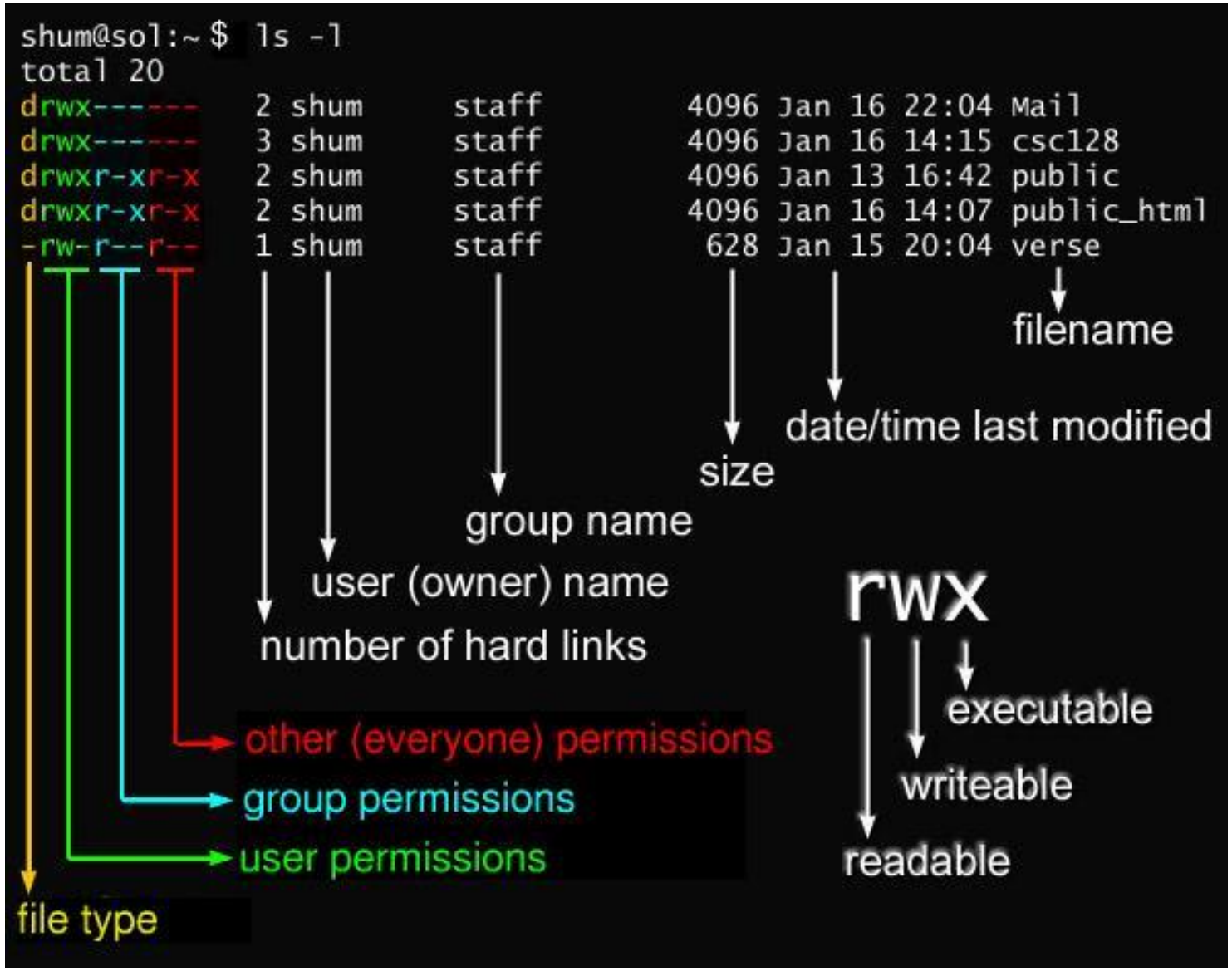
https://www.guru99.com/file-permissions.html
https://www.thegeekdiary.com/understanding-basic-file-permissions-and-ownership-in-linux/

Permissions



https://www.guru99.com/file-permissions.html
https://www.thegeekdiary.com/understanding-basic-file-permissions-and-ownership-in-linux/

Permissions



https://www.guru99.com/file-permissions.html
https://www.thegeekdiary.com/understanding-basic-file-permissions-and-ownership-in-linux/

google.com/search?q=number+of+the+hard+link+in+ls+-l&rlz=1C1CHBF_enCA917CA917&oq=number+of+the+hard+li...

Apps | fullstack | courses by subject | emails | some tools for data... | Machine Learning | Egypt radio | cordova plugin | pcb | React | ACCOUNT | Other bookmarks

Google

number of the hard link in ls -l

All | News | Images | Videos | Shopping | More | Settings | Tools

About 16,600,000 results (0.67 seconds)

2

Since /parent/dir/sub1 has no subdirectory, its **link** count is 2 (the sub1 entry in /parent/dir and the . entry in /parent/dir/sub1). To minimize the **amount** of special-casing for the root directory, which doesn't have a “proper” parent, the root directory contains a .. entry pointing to itself.

unix.stackexchange.com › questions › why-does-a-new-di...

Why does a new directory have a hard link count of 2 before ...

About Featured Snippets | Feedback

People also ask

What is number of links in LS?

What is hard link count?

Type here to search

ENG 5:18 PM 9/24/2020

Modifying Ownership and Permissions

Modifying Ownership and Permissions

chown

```
chown [OPTIONS] USER[:GROUP] FILE(s)
```

- `USER` - The specified user will become the owner of the given files
- `USER:` -The user will become the owner of the files, and the files group ownership is changed to user's login group.
- `USER:GROUP` - The user ownership of the files is changed to the given user and the group ownership is changed to the given group.
- `:GROUP` - The group ownership of the files is changed to the given group.
- `:` the user and the group, no change is made.

By default, on success, chown doesn't produce any output and returns zero.

chgrp

```
chgrp [OPTIONS] GROUP FILE..
```

```
sudo useradd nameofuser
sudo  groupadd nameofgrop
```

https://linuxize.com/post/linux-chown-command/
https://linuxize.com/post/chgrp-command-in-linux/
https://www.geeksforgeeks.org/chgrp-command-in-linux-with-examples/?ref=lbp

Modifying Ownership and Permissions

chmod

chmod [OPTIONS] [ugoa...][-+=]perms...[,...] FILE...

• u - The file owner. • g - The users who are members of the group. • o - All other users. • a - All users, identical to ugo.	• - Removes the specified permissions. • + Adds specified permissions. • = Changes the current permissions to the specified permissions. If no permissions are specified after the = symbol, all permissions from the specified user class are removed.

```
chmod 640 file
```

stat

stat -c "%a" filename

https://linuxize.com/post/chmod-command-in-linux/

Makefile

Makefile

- **Makefile** is a program building tool which runs Linux
- If you want to run or update a task when certain files are updated, the **make** utility can come in handy.
- The make utility requires a file, **Makefile**, which defines set of tasks to be executed.
- You may have used **make** to compile a program from source code.
- Most open source projects use make to compile a final executable binary, which can then be installed using make install.

```
say_hello:
        echo "Hello World"
```

makefile

project1

Run this command inside project1 directory `make`

https://opensource.com/article/18/8/what-how-makefile
https://www.tutorialspoint.com/makefile/index.htm

Makefile

- **Makefile** is a program building tool which runs Linux
- If you want to run or update a task when certain files are updated, the **make** utility can come in handy.
- The make utility requires a file, **Makefile**, which defines set of tasks to be executed.
- You may have used **make** to compile a program from source code.
- Most open source projects use make to compile a final executable binary, which can then be installed using make install.

```
say_hello:
        @echo "Hello World"
```

makefile

project1

Run this command inside project1 directory `make`

https://opensource.com/article/18/8/what-how-makefile
https://www.tutorialspoint.com/makefile/index.htm

Makefile

```c
#include <stdio.h>
int main() {
// printf() displays the string inside quotation
   printf("Hello, World!");
   return 0;
}
```

main.c

```makefile
build:
        @gcc main.c
```

makefile

project2

Run this command inside project1 directory `make; ./a.out`

https://opensource.com/article/18/8/what-how-makefile
https://www.tutorialspoint.com/makefile/index.htm

Makefile

```
#include <iostream>
using namespace std;
int main(){
cout << "Hello World!" <<endl;
 }
```

main.cpp

```
build:
        @g++ main.cpp
```

makefile

project3

Run this command inside project1 directory `make; ./a.out`

https://opensource.com/article/18/8/what-how-makefile
https://www.tutorialspoint.com/makefile/index.htm

Shell Scripts

Shell scripts
hello world

- Shell scripts are a powerful tool to automate interaction with the many utilities available on UNIX-like systems.

Basic Bash script

A shell script typically begins with a declaration of which shell should be used to interpret the script. This is often called a hashbang because it begins with the characters `#!`. For example, if the first line is `#!/bin/bash`, the script should be interpreted using the executable at `/bin/bash`.

cat / etc /shells command for all available shells.

```
#!/bin/bash
echo "Hello World"
```

Hello.sh

Shell-scripts

Run these command inside Shell-scripts directory

```
sudo chmod +x Hello.sh
./Hello.sh
```

Shell scripts
while loop

- Shell scripts are a powerful tool to automate interaction with the many utilities available on UNIX-like systems.

Basic Bash script

A shell script typically begins with a declaration of which shell should be used to interpret the script. This is often called a **hashbang** because it begins with the characters `#!`. For example, if the first line is `#!/bin/bash`, the script should be interpreted using the executable at `/bin/bash`.

```
#!/bin/bash
x=0 # Initialize x to zero
while [ $x -lt 20 ]; do # Keep
looping while x < 5
echo $x
x=$(($x+1))
done
```

whileloop.sh

$SHELL command will give the current shell
cat / etc /shells command for all available shells.

Shell-scripts

Run these command inside Shell-scripts directory

```
sudo chmod +x whileloop.sh
./whileloop.sh
```

Shell scripts
until loop

- Shell scripts are a powerful tool to automate interaction with the many utilities available on UNIX-like systems.

Basic Bash script

A shell script typically begins with a declaration of which shell should be used to interpret the script. This is often called a **hashbang** because it begins with the characters `#!`. For example, if the first line is `#!/bin/bash`, the script should be interpreted using the executable at `/bin/bash`.

```
#!/bin/bash
x=0 # Initialize x to zero
until [ $x -ge 5 ]; do # Keep
looping until x >= 5
echo $x
((x=$x+1))
done
```

untilloop.sh

$SHELL command will give the current shell
cat / etc /shells command for all available shells.

Shell-scripts

Run these command inside Shell-scripts directory

```
sudo chmod +x untilloop.sh
./untilloop.sh
```

Shell scripts
function

This creates a function named `func` which sends hello to standard out. The function may be passed arguments, which are accessed the same way as arguments to the shell script. (i.e. the function will have its own set of variables `$1`, `$2`, etc).

```
#!/bin/bash
function func
{
echo Hello $1
}
func World
```

function.sh

Shell-scripts

$SHELL command will give the current shell
cat / etc /shells command for all available shells.

Run these command inside Shell-scripts directory

```
sudo chmod +x function.sh
./function.sh World
```

Shell scripts
array

To declare an array

```
declare -a myarray
```

```
#!/bin/bash
declare -a myarray
myarray[0]="First Item"
myarray[1]="Second Item"

echo ${myarray[0]}
```

array.sh

Shell-scripts

$SHELL command will give the current shell
cat / etc /shells command for all available shells.

Run these command inside Shell-scripts directory

```
sudo chmod +x array.sh
./array.sh World
```

Shell scripts
loop over array

To declare an array

```
declare -a myarray
```

```
#!/bin/bash
x=0
while [ $x -lt 5 ]; do
read userinput
lines[$x]=$userinput
(( x = $x + 1 ))
done
y=0
while [ $y -lt $x ]; do
echo "You said ${lines[$y]}"
(( y = $y + 1 ))
Done
```

loopArray.sh

$SHELL command will give the current shell
cat / etc /shells command for all available shells.

Shell-scripts

Run these command inside Shell-scripts directory

```
sudo chmod +x loopArray.sh
./loopArray.sh World
```

Shell scripts
Using &&, ||, ; and {}

Placing the `&&` syntax between commands causes the succeeding command to be executed only of the preceding command succeeds (i.e. program returns a value of 0).

```
#!/bin/bash
cat /etc/hosts > /dev/null && echo "It worked!"
```

Placing the `||` syntax between commands causes the succeeding command to be executed only of the preceding command fails (i.e. program returns a value other than 0).

```
cat /etc/fakefile || echo "It failed!"
```

Placing `;` in between commands runs them in sequence, as if they were on separate lines.

```
echo "Part one" ; echo "Part two"
```

You can use curly braces (`{` and `}`) to group commands together. For instance, the following code will pipe the output of two commands run in sequence to `sed`.

```
#/bin/bash
{ echo "Part 1"; echo "Part 2"; } | grep “2”
```

Shell scripts
Important Notes

Whitespace

Many features in bash are whitespace sensitive. For instance, writing `x = 0` instead of `x=0` in the initialization above would result in an error. Writing `[$y -lt $x]` instead of `[ $y -lt $x ]` would also result in an error.

Indexed variables in strings

If you would like to include the value of an array element in a string, you will need to wrap the variable around the `${}` syntax, otherwise it will be interpreted literally. For instance, writing `lines[$y]` instead of `${lines[$y]}` in the above example would result in an output of the following:

Thank you